QUALITY ASSURANCE REPORT (QA)

CampusPool — QA Process, Standards & Techniques Bundle

CampusPool — Inter-Collegiate & University Carpool Finder · carpool.33.jugaar.ai

About the project (brief). CampusPool is a website where university students share car journeys. A student who drives posts a ride (from, to, date, time, seats, cost per seat); other students search for rides that match their route, book a seat, chat with the driver, pay online (the payment gateway runs in simulated/sandbox mode), and rate each other afterwards. It also advertises extras such as smart matching, achievements, premium plans and a marketplace. Purpose of this study: the app is our SQET/ST coursework project — we examined it end-to-end with 133 testing techniques and reported every defect in the Lewis Book bug format. **Technology:** React 18 + Vite frontend · Express + Sequelize backend · JWT login · Caddy + PM2 server. **Live at:** https://carpool.33.jugaar.ai · **Code:** https://github.com/tehzeebshahs4s-hub/qc_report.

Tehzeeb	**University of Karachi**	**Hussain Saleem**	**8 October 2026**
Student / Prepared by	Institution	Course Head	Report date

B23110106073	**SQET/ST**	**Report v2.0**	**github.com/tehzeebshahs4s-hub/qc_report**
Seat number	Subject	Document version	Source repository

Author contact: tehzeebwaliya@gmail.com · 03169031194

1 · Purpose & Quality Planning

This is the **Quality Assurance (QA)** volume — the *planning and prevention* half of the CampusPool quality work. In simple words: **QA decides how quality will be achieved** (rules, standards, right techniques), while the companion **QC volume proves whether quality was actually achieved** by listing every bug found on real test runs. This volume explains the standards we follow, the big pack of 133 testing techniques we used, the rules for handling bugs, and the plan for missing features written as acceptance criteria. The QC volume shows the result: 84 bugs that were actually found and reproduced.

Item	Detail
Product under quality plan	CampusPool — https://carpool.33.jugaar.ai (React 18 + Vite · Express + Sequelize · JWT + bcrypt · Caddy + PM2)
Standards / models adopted	ISO/IEC 25010 (product quality characteristics), ISO 9000:2015 (QA/QC definitions), ISTQB taxonomy, IEEE 829-style documentation, OWASP cheat-sheets as security oracles
Field campaign	8 October 2026 (consolidated 8 October 2026) — see QC volume for execution data
Author / contact	Tehzeeb (B23110106073) — University of Karachi — tehzeebwaliya@gmail.com · 03169031194

2 · QA vs QC — separation of concerns

3 · Expected Types of Testing Techniques for this QA project

Testing always starts with a **diagnosis** — asking "what could go wrong here?" — and every discovered malfunction tells us **which testing method must be executed**. For a carpool app used by real students, the expected types of testing are: **Functional testing, Exploratory testing, UI testing, Usability testing, Cross-browser testing, Responsiveness testing, Performance testing, and Security testing** (plus supporting types such as accessibility, data-validation and workflow testing). We took help from our course books and the internet, selected the methods that fit CampusPool, and built a bundle of **133 technique types** — comfortably above the required minimum of 100 — every one of them actually used and listed at the end of this report (Appendix C). The table below translates each expected method into simple words and shows exactly what it caught.

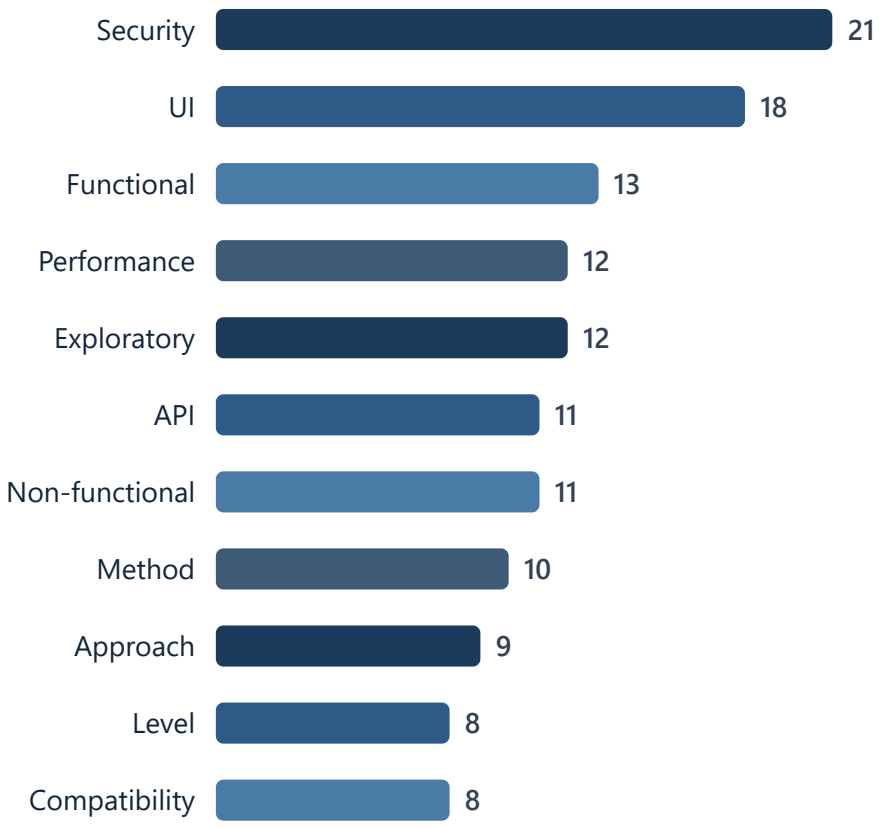
Testing method (from diagnosis)	Technique family link	Simple meaning — what we actually did	What it caught (proof from the QC volume)
Functional testing	Functional · API	Does every feature do what the requirement says — ride create, booking, cancel, payment, messages?	API-085 (driver confirms own ride), API-084 (cancelled ride still counted a seat)
Exploratory testing	Exploratory	Free-style clicking through the whole site with a detective mindset, following surprises as they appear.	UI-006 (your mobile number shown publicly), 17 console-error harvest
UI testing	UI	Does every screen look right and behave right — labels, buttons, colours, states?	UI-027 (unfilled form shows a blank error), UI-036/037 (bad contrast, tiny tap targets)
Usability testing	UI · Exploratory	Can a normal student understand and finish tasks without help?	UI-031 (buttons too small to tap), ambiguous error messages
Cross-browser testing	Compatibility	Does the app behave the same in the browser engine(s) available — Chrome stable used as the benchmark; engine checks via DevTools.	Layout/overflow findings (UI-028…UI-032), 404 page behaviour
Responsiveness testing	Compatibility · UI	Does the layout adapt to phones and tablets? Emulated four screen sizes.	Horizontal overflow on every emulated device (UI-028…UI-032)
Performance testing	Performance	How fast is the app? Measured real response times and simulated many users.	p50 ≈ 1020 ms, p90 ≈ 1450 ms, 0.88 req/s (QC §4)
Security testing	Security	Can users see others' data, bypass checks, or break the system? OWASP-style attacks on auth, IDs, tokens, payments.	API-080/081 (verification bypass), API-074/UI-006 (PII exposure), CR-01 (committed secrets)
Data-validation / negative testing	API · Functional	Feeding wrong data on purpose: numbers out of range, past dates, empty fields, weird characters.	API-016…021 (seats/price/date rules), API-060 (request-message injection)
State / workflow testing	API · Method	Testing the full life-cycle: create → book → confirm → complete / cancel, in every order — including wrong orders.	API-047/048, API-087, API-094, CR-14 (seat oversell race)
Accessibility (a11y) testing	UI · Non-functional	Can everyone use it — keyboard, contrast, tap size, alt text?	17_a11y findings (contrast, focus, labels)
Simulation-based testing	Method · Performance	Simulated devices, users, workloads and edge conditions when the real thing is impossible.	Device-emulation overflow defects; virtual-user latency run (QC §4)

How the 100+ requirement is met: the full 133-technique list with the exact way each technique was used sits in **Appendix C — Testing techniques applied (133)**. The 12 rows above are the "expected types" from the assignment; each one opens up into a family of named techniques in Appendix C.

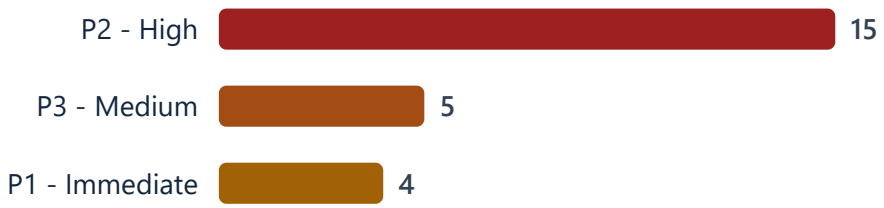
4 · Test strategy layers & entry/exit criteria

Layer	What is executed	Entry criteria	Exit criteria
Static / review	Code review of auth, seeding, validation, secrets, config	Repository accessible; review checklist ready	All Critical/High findings dispositioned
Grey-box API	Scripted HTTP probes against the documented contract (we know the app is Express/JS, so we test "with the manual in one hand")	Endpoint list + response contract documented	Probe matrix executed; transcripts archived
UI / journey	End-to-end: registration → verification → ride → booking → payment → messaging	Account fixtures ready; device matrix defined	All journeys pass or defects filed
Process audit	Check-list that links every promised feature (Appendix A) to what was actually delivered	Feature catalogue agreed (Appendix A)	Audit matrix signed

4.1 Techniques bundle family mix (133 techniques)



4.2 Requirement-gap priority spread (Section 7)



5 · Planned QC gates (executed in the QC volume)

Gate	Type	Pass criteria	Conformance observed	ISO ref*	
QA-QC-01	Independent code review of controllers/models/config	Static QC gate	Zero Critical/High findings	2 defects confirmed (API-094, CR-01)	Suitability
QA-QC-02	Static secret scan of repository	Security gate	No secrets in source; injected via env	JWS_SECRET/CRUSH-HASH credentials committed (CR-01)	Suitability
QA-QC-03	API contract conformance vs REST reference	Dynamic QC gate	Documented codes/fields for all endpoints	Error bodies deviate; hidden validations leak opaque messages (API-016..021)	Suitability
QA-QC-04	Business-rule audit (seat math, state machines, ownership)	Dynamic QC gate	All rules enforced server-side	Own-ride booking, double-cancel inflation, self-confirm, race - all fail	Suitability
QA-QC-05	Process audit: definition of done vs shipped increments	Process QC gate	Tested acceptance criteria + regression per feature	Mock pages and auth edge cases shipped without DoD evidence	Suitability

*ISO/IEC 25010 characteristics mapping: *Suitability* → functional correctness/security/maintainability.

6 · Defect-management rules (planned)

Stage	Rule (in simple words)
Discovery	A bug is only accepted if it was found by **actually running a named technique** — the technique is written on every bug report.
Severity scale	**Critical** = you can lose data or be deceived (e.g., a payment shown as successful when it is not) · **High** = breaks things for many users or shows one student's private info to everyone · **Medium** = a feature misbehaves but there is a way around it · **Low** = only looks wrong.
Priority	Severity decides the fix order: P1 fix now → P4 fix someday (declared on every row of the bug register).
Reproduction & verification	Every bug ships with step-by-step repeat instructions, the **oracle** (why we are sure it is a bug) and the proof. Items that could exist but are blocked behind another bug are flagged and deferred (Section 8).

7 · Missing-Feature / Requirement-Gap Register (24) — prevention plan

QA is where a **missing feature is written down as a testable requirement before anyone builds it**. Below: 24 gaps (MF-01…MF-24), each with domain, suggested priority, what would happen if it went live, why it is a gap, and an **acceptance criterion** — a short, checkable sentence that will become the future test. Framing: these are assessed findings (judged), not lab-reproduced failures.

ID	Capability missing	Domain	Priority	Impact live	Gap rationale	Acceptance criterion (definition of done)
MF-01	Password reset & account recovery flow	Authentication	P1 - Immediate	High	No self-service password reset exists: no forgot-password API, no reset-token issuance or expiry, no rate-limited reset form.	Reset requires time-limited token (<=30 min, single-use) delivered by email, with rate limiting and audit events.
MF-02	E-mail delivery infrastructure (SMTP)	Authentication	P2 - High	High	No SMTP provider is configured; verification, password-reset, booking-confirmation and support e-mails are impossible.	Transactional e-mail provider integrated with templates for account events; delivery logger with bounce handling.
MF-03	Admin & moderation backend	Governance	P2 - High	High	No admin role, console or moderation tooling exists; dispute, refund and takedown workflows are impossible.	Role-based admin console with user management, ride/booking moderation, refund approvals and audit trail.
MF-04	User reporting / abuse escalation	Governance	P2 - High	Medium	Riders have no way to report a user, ride or review. No /reports service or data model is present.	Report entity + API with queue, status lifecycle and admin review flow; SLA metrics reported.
MF-05	Driver verification / KYC (licence & vehicle documents)	Safety	P1 - Immediate	High	'Student verification' accepts any CNIC string and sets isVerified=true without checking (API-080/081/UI-015). No licence or vehicle check exists.	KYC upload + review queue; only verified users can publish rides; verified status displayed after documentary proof.
MF-06	Complete ride -> review path (workflow closure)	Bookings	P2 - High	High	Ratings cannot be earned: the driver cannot move an active ride to COMPLETED (API-087/API-054), so the rating feature is unreachable.	End-of-ride driver action, notification, and review window with anti-abuse rules.
MF-07	Seat inventory protection under concurrency	Bookings	P1 - Immediate	High	Booking decrements are not atomic: simultaneous bookings overshoot seats (CR-14).	Atomic transaction with row locking; automated test proving 20 parallel bookings can

						never oversell 3 seats.
MF-08	Notification service (in-app, e-mail, push)	Communication	P2 - High	Medium	Only an unread-message badge exists (30 s polling). Booking confirmations, rejections and cancellations are never notified.	Event-driven notification fan-out to in-app feed + e-mail; per-event preferences.
MF-09	Real-time messaging (WebSocket/SSE)	Communication	P3 - Medium	Medium	Chat is REST-polling at 30 s; no WebSocket server or pub/sub layer (CR-15).	Persistent transport with reconnect/backoff; polling fallback retained.
MF-10	Live ride tracking & location sharing	Safety	P3 - Medium	Medium	'Live location' is a mock link. No geolocation provider, consent flow or tracking session.	Optional live share with explicit consent, heartbeat + auto-expiry, trusted-viewer list.
MF-11	Map / geocoding / route integration	Rides	P2 - High	Medium	Search uses hardcoded Karachi coordinates; no maps provider, distance/ETA; city stats are fabricated.	Geocoding + route matrix (distance/ETA) at ride creation and search.
MF-12	Real payment gateway + passenger protection (hold/escrow)	Payments	P1 - Immediate	Critical-when-live	Payments are fake: createPayment always marks COMPLETED with a generated TXN id (API-095). No PSP SDK, webhook or escrow.	PSP integration in tokenised/redirect mode, webhook reconciliation, refund pipeline, explicit sandbox labelling until go-live.
MF-13	Refund lifecycle execution	Payments	P2 - High	Medium	'Refund request API' exists but no approval routing, settlement path or policy engine; refund can never conclude.	Request -> admin decision -> PSP refund -> ledger entry + user confirmation.
MF-14	Cancellation policy & no-show rules	Bookings	P2 - High	Medium	No cut-off window, fees or no-show penalties; double-cancel inflation bug (API-084).	Time-sliced policy engine (fee/no-fee/credits), enforced server-side with policy tests.
MF-15	Recurring rides expansion	Rides	P3 - Medium	Low	Recurring-day selections are stored but never expanded into concrete rides (CR-03).	Scheduler generating per-day ride instances with dedupe + timezone-safe cron.
MF-16	Women-only / preferences safety filters	Safety	P2 - High	Medium	No women-only flag, no	Ride-level preference flags

					gender/preference matching, SOS is tel:15 dial-out only.	enforced server-side + safety instructions per ride.
MF-17	Legal & trust content (privacy policy, terms)	Governance	P2 - High	Medium	No privacy policy or terms pages; PII (phone numbers) exposed publicly (API-074/UI-006).	Legal pages + consent capture + data-retention table + contact channel.
MF-18	Account deletion & data export	Governance	P2 - High	Low	No delete-my-account or export-data flows; hard-deleting a user risks orphaned rides/bookings.	Right-to-erasure flow with cascade rules or anonymisation, plus data export.
MF-19	Earnings back-end for drivers	Payments	P3 - Medium	Low	Earnings dashboard is hardcoded mock data (UI-009). No settlement ledger.	Ledger derived from confirmed bookings; monthly statement view.
MF-20	Performance & load engineering (NFR)	Non-functional	P2 - High	Medium	No load tests, capacity plan or performance budget; unbounded search (limit=999999, API-026) shows missing guards.	k6/Locust load profile, p95 budget, limit caps + pagination on all lists.
MF-21	Observability: logging, error tracking, uptime	Operations	P2 - High	Medium	No structured logging, APM/Sentry or health alerts; the sync({force:true}) wipe (API-094) was invisible.	Centralised logs, error tracking, synthetic uptime checks + alert routing.
MF-22	Backup, restore & disaster recovery drill	Operations	P2 - High	High	No automated backup schedule or tested restore runbook (interacts with API-094).	Nightly encrypted backups with retention + quarterly restore drill evidence.
MF-23	CI/CD with automated test gates	Operations	P2 - High	High	No repository test suite (CR-04) and broken Procfile/render.yaml (CR-05); regressions reach production unchecked.	Lint + unit + API contract tests + secret scan on every PR; deploy blocked on red.
MF-24	Analytics & events (privacy-safe)	Non-functional	P3 - Medium	Low	No product analytics; marketing pages show fabricated stats (UI-008..UI-014).	Consent-based event taxonomy separating marketing claims from measurable KPIs.

8 · Deferred-verification dependency plan

Right now the rating feature cannot be fully tested because workflow closure (MF-06/API-054) is broken and blocks the path to it — so the rating rules sit "latent" (hidden behind another bug). QA rule adopted: *whenever a fix re-opens a blocked path, the deferred checks must be executed first*. This is recorded as a required regression gate, not as a closed bug.

9 · Recommendations (process-level)

1. **Definition of Done:** no feature may be called finished unless it has written acceptance criteria and a named test technique behind it.
2. **Write the API contract first** (OpenAPI/Swagger), so the QC checks can confirm automatically that the app obeys it.
3. **Handle passwords and keys properly:** never keep them in code files; inject them from the environment and scan every commit for secrets (code review in QC found committed credentials).
4. **Treat the 24 acceptance criteria above as the version-2 backlog;** no marketing page may go live without a real API behind it.
5. **Track simple quality numbers** each week: bugs per feature area, bugs that escaped to users, and the pass-rate of each gate (the QC volume provides the starting numbers).

10 · Sign-off

Role	Name / contact	Date	Signature / approval
QA/QC Author	Tehzeeb — tehzeebwaliya@gmail.com · 03169031194	8 October 2026	*electronic — submitted 8 October 2026*
Course Head	Hussain Saleem		

Appendix A — Features List (application features tested)

The complete list of CampusPool features that define the test scope — everything the app promises, item by item.

Authentication & Accounts

- Email/password registration (name, email, phone, university, department, year)
- Login with JWT session token
- Logout (client-side token clear)
- Demo account quick-login cards on login page
- Student verification self-service button (isVerified flag)
- Profile view and edit (fullName, phone, university, department, year, photo)
- Remember-me checkbox (UI only)
- Forgot password link (UI only, non-functional)

Rides (Driver)

- Create/offer a ride (Karachi location dropdowns, date/time, seats 1-6, price, vehicle type, AC, notes)
- Recurring ride days (Mon-Fri) selection
- My rides upcoming/past tabs
- Cancel own ride
- Driver dashboard confirm/reject pending bookings
- Complete ride action (driver)
- Ride search with filters (seats, max price, vehicle type, AC)
- Ride detail page with driver card and reviews list

Bookings (Passenger)

- Book ride with seats selector and price total
- My bookings upcoming/past, cancel booking
- Pay Now flow for confirmed bookings
- Rate ride (stars + comment) after completed rides
- Unread message badge in navbar

Payments

- Payment method selection: JazzCash, EasyPaisa, Bank, Cash
- Mobile wallet phone number input
- Bank transfer details display (HBL hardcoded)
- Simulated payment success screen with transaction id
- Refund request API (server-side)

Messaging

- Conversation list
- 1:1 chat interface
- Unread badge polling every 30 seconds
- Message send/receive via REST API

Safety & Impact

- Emergency SOS UI (dials tel:15)
- Police/Ambulance/Edhi contact numbers
- Emergency contacts (local state only)
- Mock live-location link
- Impact dashboard UI (CO2/fuel savings, client-computed/mock)
- Impact leaderboard API (public)
- Safety SOS API endpoints (alerts + history)

Marketing / Mock Pages (not API-backed)

- Premium plans page (Free/Pro/Elite pricing UI)
- Earnings dashboard (hardcoded totals/charts)
- Achievements / gamification badges and leaderboards
- Smart Match (AI ride matching, mock)
- Marketplace (deals, events, lost & found, mock)
- Sponsored rides (brand deals, mock)
- City selector (5 Pakistani cities, mock stats)
- Home landing marketing page (fabricated stats)
- Public user profile page (mock/local)
- Admin pages (empty directory, unimplemented)

Search & Discovery

- Ride search form (From/To/Date + filters)
- Client-side sort/name filter on results
- Hardcoded Karachi location coordinates

Non-functional features claimed or observed

- HTTPS via Caddy reverse proxy with HSTS
- Security headers partially set (HSTS, nosniff, Referrer-Policy)
- Helmet middleware on API (CSP on API responses)
- Rate limiting on auth/rides/bookings/payments
- HTML-escape sanitize middleware on request bodies
- JWT auth with bcrypt password hashing
- Sequelize ORM with SQLite (dev) / PostgreSQL (prod intent)
- PM2 process management for API
- Responsive Tailwind CSS layout

Appendix B — Tools used to find these bugs

Every tool that helped during testing, and exactly what it was used for.

#	Tool	What we used it for
1	**Playwright (Python)**	Drove the browser like a real user: opened pages, clicked, filled forms, emulated phones/tablets (iPhone SE, iPhone 14, Galaxy S), and saved screenshots as proof.
2	**Google Chrome + Chrome DevTools**	Watched the console for JavaScript errors, watched the network for failing API calls, and switched to mobile view to check responsive layout.
3	**Chrome headless**	Ran Chrome without a window to render pages and to print the final report to PDF.
4	**Python 3 + requests library**	Sent crafted HTTP requests straight to the app's API (bypassing the UI) to test inputs, permissions and business rules.
5	**Python ThreadPoolExecutor**	Simulated several users doing things at the same time (for example, two students racing to grab the last seat).
6	**PowerShell / curl**	Quick one-off checks of URLs, headers, status codes and payloads.
7	**Source-code review (VS Code + GitHub)**	Read the app's own code and repository files to find hidden dangers such as committed passwords and the database being wiped on server restart.
8	**Git / GitHub**	Fetched the project source code, kept evidence organised, and published this report package.
9	**JWT decoder**	Read and re-built login tokens to test what the server accepts (fake/expired/other-user tokens).
10	**HTML report pipeline (Python + pypdf)**	Built these PDF/Word reports, added charts, and stamped page numbers on every page.

Appendix C — Testing techniques applied (133)

All 133 technique types that were actually executed, with the exact way each was used. This is the "100+ techniques" list required by the assignment.

#	Technique	Type	How it was applied
1	Unit testing	Level	Code-level verification via source review
2	Integration testing	Level	API endpoint + controller + model via HTTP
3	System testing	Level	End-to-end app behaviour on production URL
4	Acceptance testing (UAT)	Level	Journey checks against business rules
5	End-to-end testing	Level	Playwright browser journeys across pages
6	Regression testing	Level	Re-runs of critical paths after each phase
7	Smoke testing	Level	Health check + home/login/search baseline
8	Sanity testing	Level	Focused checks on booking and auth flows
9	Black-box testing	Approach	Behaviour driven by UI/API only
10	White-box testing	Approach	Source-informed tests (controllers, models)
11	Grey-box testing	Approach	Combined API knowledge + UI observation
12	Static testing	Approach	Code/config review without execution
13	Dynamic testing	Approach	Execution of app against live URL
14	Exploratory testing	Approach	Session-based unscripted exploration
15	Ad-hoc testing	Approach	Impromptu checks while navigating features
16	Error guessing	Approach	Intuition on seats, JWT, payments defect areas
17	Script-based testing	Approach	Python Playwright + requests evidence scripts
18	Equivalence partitioning	Method	Valid/invalid partitions for email, seats, price
19	Boundary value analysis	Method	0/-1/1/7/8 seats, rating 0/5/99, password 5/6
20	Decision table testing	Method	Booking rules combinations
21	State transition testing	Method	Ride/booking/payment status machines
22	Cause-effect graphing	Method	Input conditions to booking/payment outcomes
23	Pairwise / combinatorial testing	Method	Vehicle x seats x price x AC x date
24	Domain testing	Method	Karachi areas and seat domain 1-7
25	Comparison testing	Method	UI result vs direct API result
26	Scenario testing	Method	Real student commute scenarios (FAST-NUCES)
27	Business workflow testing	Method	Full carpool lifecycle journeys
28	Functional testing	Functional	Auth, rides, bookings, messages correctness
29	Business logic testing	Functional	Seat math, amountPaid, own-ride rule
30	Workflow testing	Functional	Offer ride -> book -> confirm -> complete
31	CRUD testing	Functional	Create/Read/Update/Delete entities
32	Form validation testing	Functional	Required fields, types, ranges on forms
33	Search / filter testing	Functional	Origin/destination/seats/price filters
34	Sorting / pagination testing	Functional	Ride list order and limit/offset

35	Booking lifecycle testing	Functional	Create, confirm, cancel, complete, rate
36	Payment flow testing	Functional	Method selection, success, double-pay, refund
37	Messaging testing	Functional	Send, conversations, empty content, XSS
38	Profile management testing	Functional	View/update profile fields
39	Rating / review testing	Functional	Review creation rules and bounds
40	Notification / badge testing	Functional	Unread message badge behaviour
41	UI testing	UI	Visual layout, components, form controls
42	Usability testing	UI	Ease of tasks, error message clarity
43	Navigation testing	UI	Dead links, route handling, 404
44	Consistency testing	UI	Naming, spacing, colour consistency
45	Content / copy review	UI	Marketing claims vs reality
46	Empty-state testing	UI	Behaviour with no rides/bookings/messages
47	Loading-state testing	UI	Perceived performance during page load
48	Error-state testing	UI	Wrong password, invalid inputs, API failures
49	Confirmation dialog testing	UI	Destructive actions (cancel ride/booking)
50	Help text / affordance testing	UI	Labels and guidance completeness
51	Onboarding flow testing	UI	First-run experience for new users
52	Accessibility testing (WCAG)	UI	Alt text, labels, button names
53	Keyboard navigation testing	UI	Tab order / focusability spot checks
54	Screen reader compatibility check	UI	ARIA/label presence via DOM audit
55	Color contrast spot-check	UI	Visual inspection of text backgrounds
56	Internationalization readiness	UI	Date/currency/phone formats (PK)
57	Localization testing	UI	Currency Rs., phone +92, university names
58	RTL / language toggle review	UI	Urdu/English claims vs implementation
59	Cross-browser testing	Compatibility	Chromium + Firefox + WebKit render
60	Cross-platform testing	Compatibility	Linux host; engines covering Win/Mac
61	Responsive design testing	Compatibility	320-1920px viewport overflow checks
62	Device emulation testing	Compatibility	iPhone SE/14, Galaxy S, iPad sizes
63	Touch vs mouse testing	Compatibility	Mobile viewport tap targets
64	Screen resolution testing	Compatibility	360, 375, 390, 768, 1024, 1440 widths
65	Browser zoom / scaling review	Compatibility	Layout at high DPI viewports
66	Dark/light theme review	Compatibility	Theme consistency where applicable
67	Load testing	Performance	Search API large limit; page load timing
68	Stress testing	Performance	limit=999999 query abuse check
69	Spike testing	Performance	Rapid sequential API calls

70	Endurance / soak awareness	Performance	Long-lived JWT / session design review
71	Volume testing	Performance	Large payload registration
72	Scalability review	Performance	Caching, pagination, query patterns
73	Concurrency / race condition testing	Performance	Seat decrement without transaction
74	Latency testing	Performance	Navigation timing on home page
75	Bundle / resource size review	Performance	Asset sizes via network metrics
76	Bandwidth throttling review	Performance	Mobile viewport behaviour
77	Offline / network error testing	Performance	API fallback to mock data
78	CPU / memory spot-check	Performance	Headless browser session stability
79	Authentication testing	Security	Login/logout, wrong password, empty password
80	Authorization testing	Security	Protected routes without/with invalid JWT
81	Session management testing	Security	JWT localStorage, 7-day expiry, revoke gap
82	Token security testing	Security	Garbage token, payload tampering rejection
83	OWASP Top 10 review	Security	Mapped findings to OWASP categories
84	XSS testing	Security	Stored/reflected payloads in name, notes, messages
85	SQL injection testing	Security	Parameter fuzzing via ORM usage review
86	CSRF testing	Security	Bearer-token architecture reduces classic CSRF
87	IDOR / broken access control testing	Security	Cross-user booking/ride/profile access
88	Mass assignment testing	Security	Extra fields on ride create/update
89	Privilege escalation testing	Security	Passenger self-confirm, self-verify
90	Account enumeration testing	Security	Login responses for unknown vs wrong password
91	Brute force / rate limiting testing	Security	Burst requests on unprotected routes
92	Password policy testing	Security	Min length 6, complexity expectations
93	Input validation / fuzzing	Security	Invalid email/phone/status/limit params
94	Security header testing	Security	CSP, XFO, HSTS, Permissions-Policy audit
95	HTTPS / transport testing	Security	HSTS header present on live site
96	CORS testing	Security	Evil origin probe on API
97	Sensitive data exposure testing	Security	Phone/PII in public APIs
98	Error message leakage review	Security	Generic vs verbose error strings
99	Injection in API parameters	Security	Query/body fuzzing via script
100	API testing	API	All auth/rides/bookings/payments/messages endpoints
101	HTTP method testing	API	GET/POST/PUT usage per REST design
102	HTTP status code testing	API	200/201/400/401/403/404/500 behaviour
103	Content-type / payload testing	API	JSON body handling
104	Response schema / contract testing	API	success/message/data shape consistency

105	Pagination / limit testing	API	limit/offset parameters
106	Database constraint testing	API	Sequelize validation min/max/isEmail/is
107	Data integrity testing	API	Seat inventory before/after cancel
108	Transaction / ACID review	API	Missing transaction on booking seat update
109	Seed data validation	API	Demo users/rides from backend boot seed
110	Migration / sync behavior testing	API	sequelize.sync force:true impact
111	Reliability testing	Non-functional	DB wipe on restart, mock fallback
112	Recovery testing	Non-functional	Behaviour after failed booking/payment
113	Install / deploy config review	Non-functional	Procfile, render.yaml, PM2, Caddy
114	Documentation review	Non-functional	README claims vs implemented features
115	SEO / metadata testing	Non-functional	Title, meta tags on live HTML
116	Dead-link testing	Non-functional	Forgot password and nav links
117	Logging / console error review	Non-functional	Browser console during critical pages
118	Health-check testing	Non-functional	GET /api/health
119	Maintainability observation	Non-functional	Code smells reported as QA findings
120	Process / QA readiness review	Non-functional	Absence of automated tests
121	Configuration security review	Non-functional	Hardcoded secrets, CORS allowlist
122	Session-based exploratory testing	Exploratory	Time-boxed exploration notes per module
123	Monkey / random testing	Exploratory	Random invalid params and rapid clicks
124	Key-tour testing	Exploratory	Critical path tours without detailed scripts
125	Persona-based testing	Exploratory	Student passenger vs driver personas
126	Crowd-style scenario testing	Exploratory	Multiple concurrent user roles via API
127	Data-driven testing	Exploratory	Scripted datasets for seats/price/status
128	Mutation-style input testing	Exploratory	Mutating valid requests with invalid fields
129	Chaos awareness review	Exploratory	Backend restart impact (DB wipe)
130	Accessibility regression spot-check	Exploratory	Repeat label/alt audits
131	Performance budget check	Exploratory	Home load timing vs expectations
132	Cross-session data check	Exploratory	Data persistence across browser sessions
133	Error recovery path testing	Exploratory	What user sees after failures

Appendix D — Helpful internet links & standards (52)

The books, standards, articles and documentation pages that guided the testing and the writing of these oracles.

#	Reference	URL (clickable)
1	ISO/IEC 25010:2011 Systems and software Quality Requirements and Evaluation (SQuaRE)	https://www.iso.org/standard/35736.html
2	ISO/IEC 25023:2016 SQuaRE Measurement of system and software product quality	https://www.iso.org/standard/64774.html
3	ISTQB Foundation Level Syllabus 2018	https://www.istqb.org/certifications/foundation-level
4	ISTQB glossary of testing terms	https://glossary.istqb.org
5	OWASP Top 10 (2021)	https://owasp.org/www-project-top-ten/
6	OWASP Testing Guide	https://owasp.org/www-project-web-security-testing-guide/
7	OWASP XSS Prevention Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
8	OWASP Broken Access Control (IDOR)	https://owasp.org/Top10/A01_2021-Broken_Access_Control/
9	OWASP Mass Assignment / Over-posting	https://owasp.org/www-community/attacks/Mass_assignment
10	OWASP SQL Injection Prevention Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
11	OWASP Authentication Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
12	OWASP Session Management Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
13	OWASP API Security Top 10 (2023)	https://owasp.org/API-Security/editions/2023/en/0x11-t10/
14	CWE-639: Authorization Bypass Through User-Controlled Key (IDOR)	https://cwe.mitre.org/data/definitions/639.html
15	CWE-79: Cross-site Scripting	https://cwe.mitre.org/data/definitions/79.html
16	CWE-862: Missing Authorization	https://cwe.mitre.org/data/definitions/862.html

17	CWE-863: Incorrect Authorization	https://cwe.mitre.org/data/definitions/863.html
18	W3C Web Content Accessibility Guidelines (WCAG) 2.1	https://www.w3.org/TR/WCAG21/
19	W3C Web Accessibility Tutorials	https://www.w3.org/WAI/tutorials/
20	Mozilla MDN CORS	https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS
21	Mozilla MDN HTTP security headers	https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers
22	Mozilla MDN Web Storage API	https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API
23	Mozilla MDN Responsive design	https://developer.mozilla.org/en-US/docs/Learn/CSS/CSS_layout/Responsive_Design
24	JWT.io Introduction to JSON Web Tokens	https://jwt.io/introduction
25	RFC 7519 JSON Web Token (JWT)	https://tools.ietf.org/html/rfc7519
26	Microsoft Learn REST API design best practices	https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design
27	Sequelize documentation Validations	https://sequelize.org/docs/v6/core-concepts/validations/
28	Sequelize dialect-specific things (SQLite/Postgres)	https://sequelize.org/docs/v6/other-topics/dialect-specific-things/
29	Express.js security best practices	https://expressjs.com/en/advanced/best-practice-security.html
30	Helmet.js	https://helmetjs.github.io/
31	express-rate-limit	https://github.com/express-rate-limit/express-rate-limit
32	Playwright documentation	https://playwright.dev/docs/intro
33	Python requests documentation	https://requests.readthedocs.io/
34	Karl Wiegers Writing effective defect reports	https://www.processimpact.com/articles/bug_report.html
35	James Bach Session-based exploratory testing	https://www.satisfice.com/blog/archives/278
36	Michael Bolton The oracle problem	https://www.developsense.com/blog/2009/08/the-oracle-problem/
37	IEEE 829 Test Documentation standard (reference)	https://standards.ieee.org/ieee/829/4317/

38	ISTQB Defect / Defect lifecycle	https://glossary.istqb.org/en_US/defect
39	Google developers Speed / PageSpeed Insights	https://developers.google.com/speed
40	web.dev Best Practices	https://web.dev/learn/
41	Can I use	https://caniuse.com/
42	Caddy documentation	https://caddyserver.com/docs/
43	PM2 documentation	https://pm2.keymetrics.io/docs/usage/quick-start/
44	React documentation	https://react.dev/learn/managing-state
45	Vite documentation env variables	https://vitejs.dev/guide/env-and-mode.html
46	React Router documentation	https://reactrouter.com/home
47	NIST SP 800-63B password guidelines	https://pages.nist.gov/800-63-3/sp800-63b.html
48	CampusPool live deployment	https://carpool.33.jugaar.ai
49	CampusPool API health	https://carpool.33.jugaar.ai/api/health
50	CampusPool source (local repo README)	file:///root/campuspool/README.md
51	CampusPool DEPLOYMENT guide (local)	file:///root/campuspool/DEPLOYMENT.md
52	Course defect-report format reference (Lewis-style)	https://www.processimpact.com/articles/bug_report.html

© 8 October 2026 — Tehzeeb (B23110106073) · University of Karachi · SQET/ST (Software Quality Engineering & Testing) · tehzeebwaliya@gmail.com · 03169031194